what is Linux

Linux is a free, OS operating system kernel that Powers million of devices - from servers to smartphones

1) open source → the source code is freely available to view, modify & distribute; Built & maintain by global community
2) multi user
3) multi tasking
4) secure
5) stable

Linux File S/m

Linux uses one main file s/m starting from the root directory /. All files & folders are connected under this root. Storage devices like USB drives & hard disks are mounted inside. This single tree structure

Key Linux Directories

* /home → stores personal files of users
* /etc → " s/m configuration files
* /bin → contains important basic commands
* /usr → " user programs, libraries & documents
* /opt → stores optional 3rd-party s/w
* /var → " changing data like logs & cache
* /tmp → " temporary files

Terminal Basic & Navigation Commands

Terminal is a text-based screen used to control Linux by typing commands. The shell reads the cmd & gives the result.

* pwd - shows the current folder path
* ls - lists files & folders
* ls -l - shows detailed file info
* ls -a - " hidden files also
* cd - moves to another folder
* cd .. - goes back one folder
* cd ~ - " to the home "
* Clear - clears the terminal screen.

Directory & file cmds

* mkdir - Creates a new folder
* rmkdir - Removes an empty "
* tree - shows folders in a tree format
* find - searches for files & folder
* locate - quickly finds files using a database
* Touch - Create a new empty file
* nano - opens a file in a text editor
* cat - Displays file content
* less - opens a file for scrolling & reading
* head - show the 1st line of the file
* tail - show " last " " "
* cp - copies files or folders
* mv - moves or renames files
* rm - delete files

Careful while using rm -r because it can delete folders and all their contents

File permissions & user mangement

Linux permission control who can read, write, or run a file

* r means read permission
* w " write "
* x " execute "

permission are set for the ~~user~~ owner, grp, & others

* chmod - changes file permission
* chown - " " owner
* ls -l - Displays file permission
* whoami - show the current username

passwd - changes a password
sudo - Runs commands with administrator permission
groups - shows the user's groups
id - shows user & group ID details

useful Linux cmds & Redirection

* date - shows date & time
* cal - show Calender
* history - shows previously used cmd
* echo - Prints text on the screen
* man - opens help info for a cmd
* which - shows the location of a cmd
* uname -a - shows s/m info
* df -h - shows disk space
* free -h - shows RAM usage

Redirection sends cmd o/p somewhere else:

* > - saves o/p to a file & replaces old content
* >> - adds o/p to the end of a file
* < - takes i/p from a file
* | - send o/p of one cmd to another cmd
* tee - shows o/p on screen & saves it to a file

Raspberry Pi Configuration, GPIO & Sensors

Sudo raspi-config is used to configure Raspberry Pi setting such a wifi, SSH, camera, SPI, I2C, UART & host name

GPIO pins allows Raspberry Pi to connect with

electronic components. using python we can turn an LE
On & off through a GPIO pin

Common components are:

LED - Gives light o/p
Push Button - Gives i/p when pressed
LDR
DHT11
Ultrasonic sensor
servo motor
Relay - Control high Power devices
LCD
RFID - Reads RFID Cards or tag

Package management, Process management & N/w Cmd

Package cmds are used to manage s/w:

* sudo apt update - updates the package list
* sudo apt upgrade - upgrade installed s/w
* sudo " install - installs new s/w
* " " remove - Removes s/w
* " " autoremove - " unnecessary packag

Process Cmds:-

* top / htop - shows running Process
* ps - Lists processes
* kill / killall - stop a Process
* reboot - Restart the s/m
* shutdown - Turns off

N/w Cmd :-

* hostname - show device name
* " -I - show IP address
* ping - Tests n/w connection
* if config / ip addr - shows n/w details
* ssh - Connects securely to another computer
* scp - Copies files securely b/w computer

Linux OS

what is OS

operating s/m is a s/w that acts as an interface b/w user & h/w

main function

* manages CPU * manages memory * manages storages & files * controls application * Manages users & devices

without OS : CPU, keyboard, display & application cannot work properly

Application → Operating s/m → H/w

what is Linux

Linux is an open-source os kernel that manages h/w resources & allows s/w to run efficiently

* Linux is open-source

hon we can turn an LED

pressed

devices

tag

gement & N/w Cmd

age s/w:

es the package list
e installed s/w
lls new s/w
ves s/w
unnecessary packages

Process

N/w Cmd :-

* hostname - show device name
* " -I - show IP address
* Ping - Tests n/w connection
* if config / ip addr - shows n/w details
* ssh - Connects securely to another computer
* scp - Copies files securely b/w computer

Linux OS

what is OS

operating s/m is a s/w that acts as an interface b/w user & h/w

main function

* manages CPU * manages memory * manages storages & files * controls application * Manages users & devices

without OS: CPU, keyboard, display & application cannot work properly

Application → operating s/m → H/w

what is Linux

Linux is an open-source os kernel that manages h/w resources & allows s/w to run efficiently

* Linux is open-source
* Linux itself is mainly the kernel

Ubuntu, Debian, Fedora, yocto Linux, Raspberry Pi OS
Linux distributions

Linux Distribution → Linux Kernel → H/w

what is Kernel?

It is the core part of an OS

Responsibility

* Process management - manages running program
* Memory " - " RAM
* Device Drivers - commu with h/w
* scheduling - decides which process runs
* File s/m - manage files
* security - control access
* N/w - handles n/w commu

User program → shell → Kernel → H/w

why Linux?

Linux is widely used because it is

1) open source - can be modified & customized
2) stable - can run for long time
3) secure - permissions & security features
4) lightweight - work with low memory
5) Highly customizable - unwanted packages can be removed
6) Strong community - large developer support

where is Linux used?

* server & cloud
* Embedded devices
* Industrial s/m

* automotive
* education
* medicine

API → Application Programming Interface
GPU → Graphics Processing unit
RISC - Reduced Instruction set computer
PIC → pheriferals Interface Controller
ARM → Advanced RISC Machine
SD → secure Digital
UART → universal Asynchronous Rx/Tx
SPI → serial peripheral Interface
I²C → Inter Integrated circuit
RTC → Real Time clock
MSSP → Manged security service Provider
IDE → Integrated Development Environment
GPIO → General purpose i/p / o/p
TRIS → Tri state
DMA → Direct memory Access
HTTP → Hyper text transfer Protocol
MQTT → message Queueing Telemetry Transport
AWS → Amazon web service
TCP → Transmission control protocol
UDP → user Datagram "
BMS → Battery management s/m
ADAS → Advanced Driver Assistance s/m
AutoSAR → Automatic open s/m Architecture
MISRA → motor Industry s/w Reliability Association

CAN → Control Area N/W

SOC → S/m on chip

IFTT → If this that

SMTP → Simple mail Transfer Protocal

~~API → Application prog~~

HTML → Hyper Text markup language

IPC → Inter process commun

JSON → Java script object notations

SQL → Structured Query language

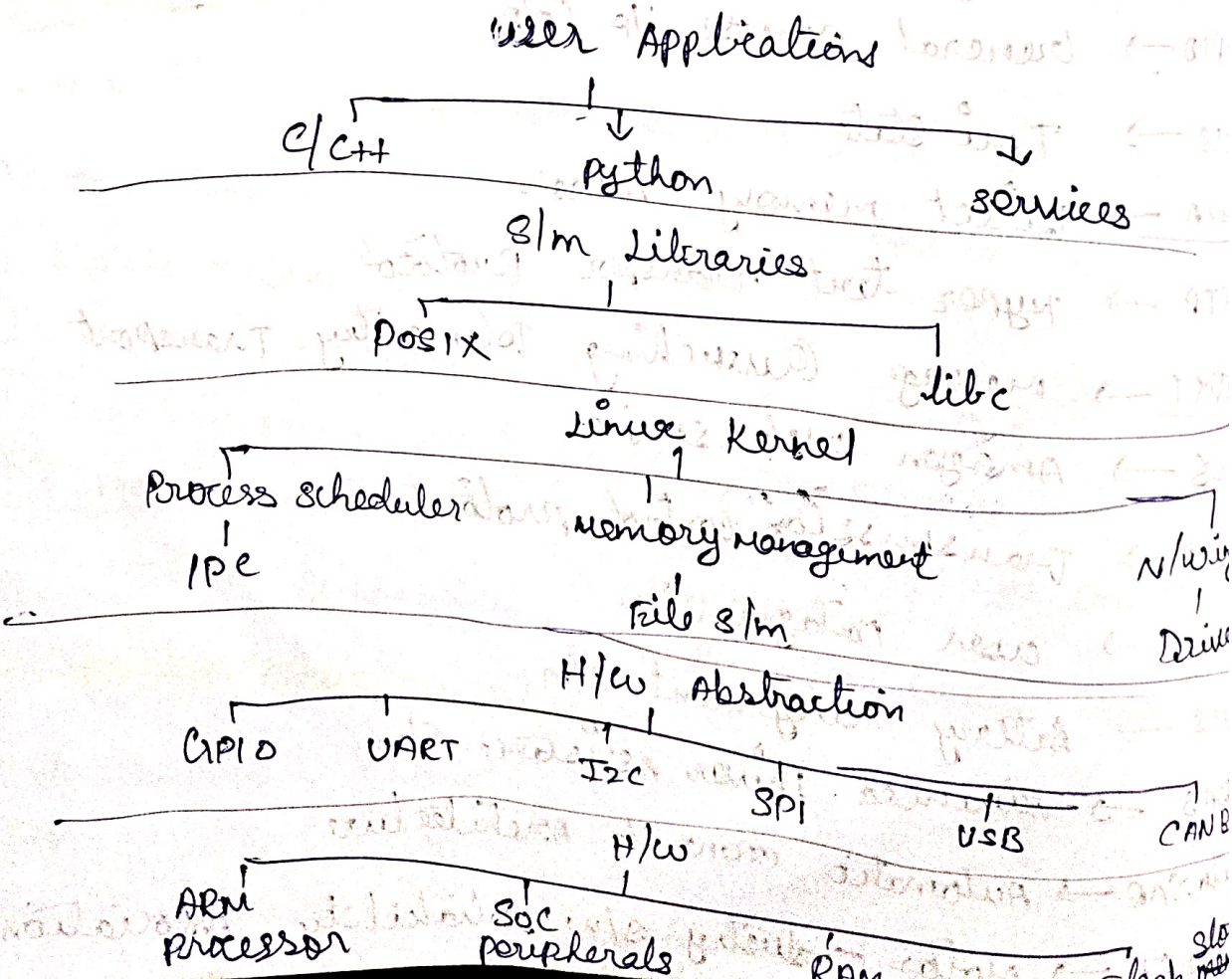
POSTX → portable operating S/m Interface

RCC → Reset Clock control

SDA → serial Data Line

SCL → serial Clock "

Embedded Linux Architecture



Important components

Bootloader
↓
Linux kernel
↓
Root file s/m
↓
Device Drivers
↓
Applications

Details:-

H/w → CPU, RAM, Storage, Peripheral.

Bootloader → initializes h/w & loads kernel

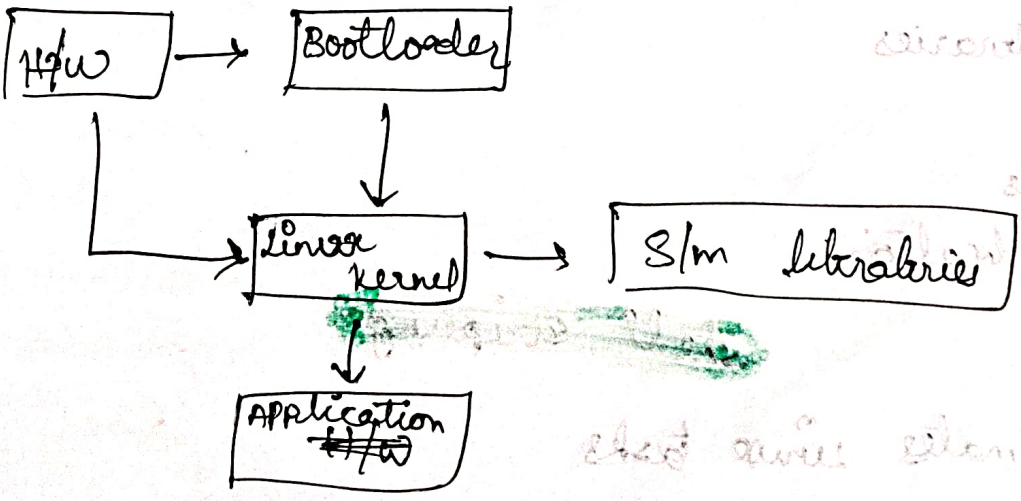
kernel → manages h/w & s/m resources

Libraries → provide programming interface

Applications → user-level programs

Short notes:-

The Linux kernel acts as the bridge s/w Application & h/w. Each layer has a specific responsibility



Typical bootloader responsibilities

* Initialize h/w
* Initialize RAM
* Load Kernel
* pass Kernel parameters
* Load Device Tree
* start Linux Kernel.

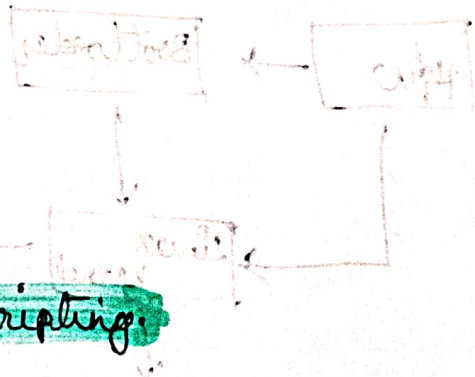
Important Features

multitasking
multi-user support
n/wing
H/w abstraction
open-source

Linux is modular & OS. This makes it especially Embedded s/m. because ~~business~~ unnecessary components be removed.

major component

Linux Kernel
Device Drivers
s/m libraries
shell
utilities
user Applications



Shell scripting:

* automates Linux tasks
* uses commands in sequence
* Common shell : Bash

* scripts usually use .sh extension

advantages

* Automation
* Easy configuration
* S/m monitoring
* ~~Easy~~ starter scripts

Ex:

```
#!/bin/bash
echo "Hello Embedded S/m"
ls
pwd
```

shell scripts help automate repetitive tasks & are commonly used for S/m initialization & configuration

Environment Configuration

It help. programs find tools, libraries & configuration information

Toolchain Setup

In embedded development, we often compile programs on a powerful PC & run them on a different processor or Embedded board. It is a collection of development tools

Main components:

Compiler
Linker
Assembler
Debugger
Libraries
Build utilities

Ex compiler:

GCC
GNU make
GDB

Cross Compilation

Host PC (x86-64) → Cross compiler (Ex: arm-linux-gcc) → Embedded target (ARM Cortex-A)

Linux processes

It process represents an executing program, & Linux provides mechanisms to create, manage & terminate processes

Important Cmd:

Ps, top, kill

Linux Kernel

Architecture

Application
↓
S/m calls
↓
Kernel
↓
H/w

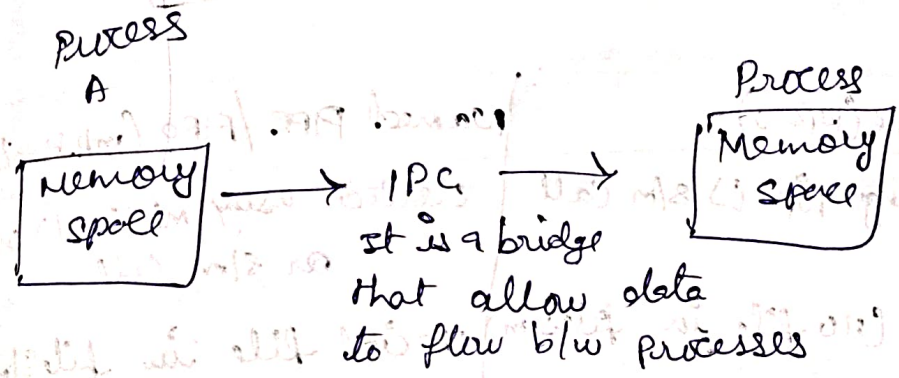
Build Process

Kernel configuration allows developers to include only the features & drivers required by the Em S/m

IPC

Inter-Process Commun allows separate Processes to Exchange data & Cordinate with each other

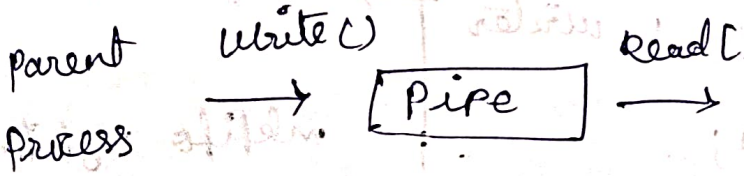
* each Process has its own memory space
* Processes needs a mechanism to share info
* IPC enables data exchange & coordination



1) Pipe
2) FIFO
3) Shared memory
4) message Queue
5) unix Domain socket

PIPE

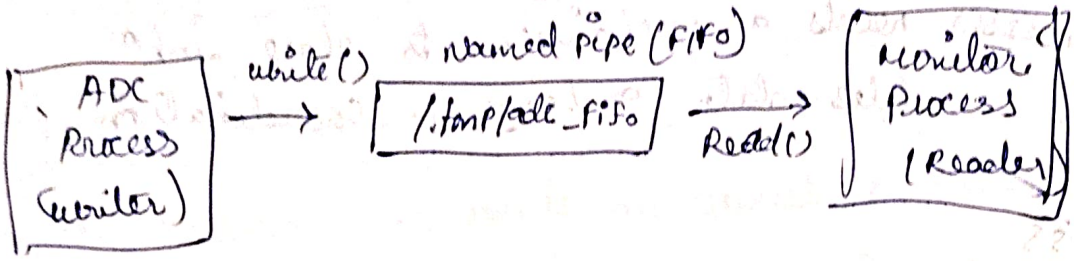
It is a kernel-managed commu channel. Data written by one process flows through the pipe & is Read by another Process



Pipe()
Create the Pipe
↓
fork()
Create child process
↓
Parent --- write() → PIPE → Read() --- → Child
↓
close unused ends

FIFO - Named PIPE

Unlike an unnamed pipe, a FIFO has a name in the file s/m, so two completely independent programs can communicate.



Feature	unnamed Pipe (Pipe())	Named PIPE / FIFO (mkfifo())
1) Creation	Created using Pipe() s/m call	Created using mkfifo() cmd or s/m call
2) Type	In-memory (no file in files/m)	special file in files/m
3) Location	Exists only in memory	Exist as " " "
4) usage	B/w related processes (Ex. Parent, child)	B/w unrelated Process
5) Persistence	Exist until processes clos the Pipe	Exist until Explicitly remove (rm)
6) Direction	unidirectional	"
7) visibility	not visible in files/m	visible
8) Blocking Behaviour	writer blocks if no reader Reader " " writer	writer " " Reader " "
9) Ex	Pipe (fd);	mkfifo myfifo

Shared memory

It allows two or more processes to access the same memory region. Instead of copying data b/w processes via kernel, both processes access the same memory directly

without shared memory

Process A ↔ Kernel ↔ Process B
(Copy) (Copy)

with shared memory

Process A (Producer) ↔ shared memory ↔ Process B (Consumer)

ADC-725

POSIX shared memory

It allows unrelated processes to share a memory region using a name in the file s/m

Common s/m calls used:

1) shm_open() 2) ftruncate() 3) mmap() 4) shm_unlink()

1) Create/open shared memory shm_open() → set size ftruncate() → map to process address space mmap() → share & comm Read/write → Cleanup shm_unlink()

shm_open() → Creates or open a POSIX shared memory object identified by a name in the file s/m

int shm_open(const char *name, int oflag, mode_t mode);

name → must start with '/' (eg. "/adc_shm")

oflag → O_CREAT - create if it does not exist
O_EXCL - Fail if it already exists
O_RDWR - open for Read & write
O_RDONLY - open for read only
O_WRONLY - " " write "

mode — use octal values (e.g. 0666)

ftruncate() - it sets the size of the shared memory object that was created or opened using shm_open()

```
int ftruncate (int fd, off_t length);
```

fd — Returned by shm_open()

length - size to set for shared memory in bytes

mmap() - that shared memory object into the calling process address space, so that the process can read & write the data like a normal memory.

```
void *mmap(void *addr, size_t length, int prot,
           flags, int fd, off_t offset);
```

Prot → PROT_READ - Read permission
" _WRITE - write "
" _READ/PROT_WRITE - Read & write

flags - mapping flags
MAP_SHARED - share changes b/w processes

offset - offset in shared memory object

Beyond pipes & shared memory

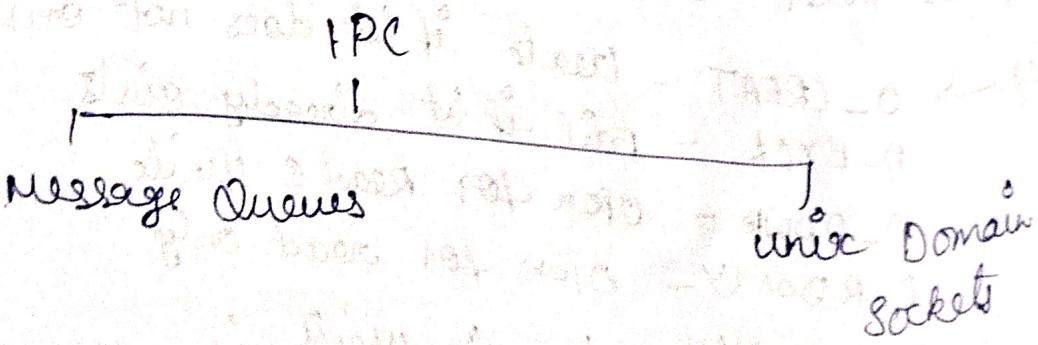
Beyond pipes & shared memory

Linux provides several IPC mechanism

pipes → simple data stream

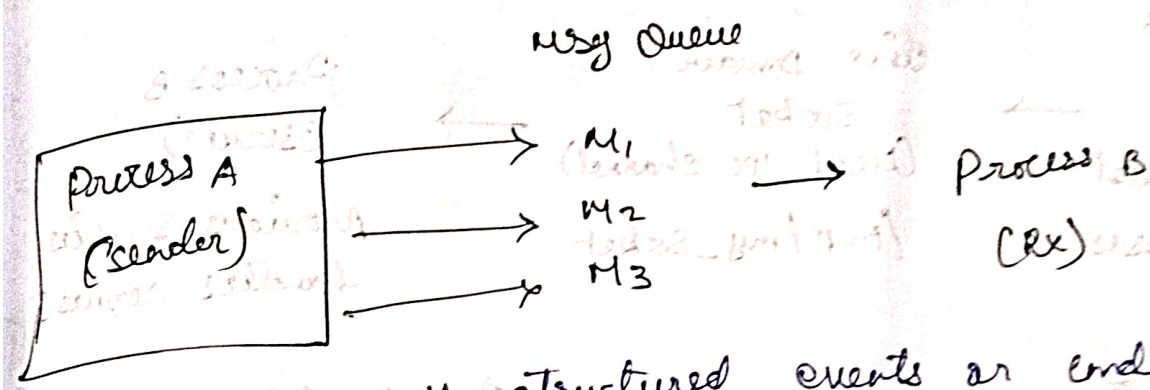
FIFOs → named comm channel

shared memory → common memory region



Message Queue

It is a kernel-managed IPC mechanism where processes exchange individual msg. Unlike a pipe, which behaves like a continous byte stream, a msg queue preserves msg boundaries.



It exchange the structured events or ends in a reliable & Organised way.

what can be sent-

* sensor readings
* control cmds
* status notifications
* Events
* Error msg.

POSIX msg Queue functions

It provides a set of sys calls to create, use & manage msg queues. these functions allow processes to send & Rx msg safely & efficiently

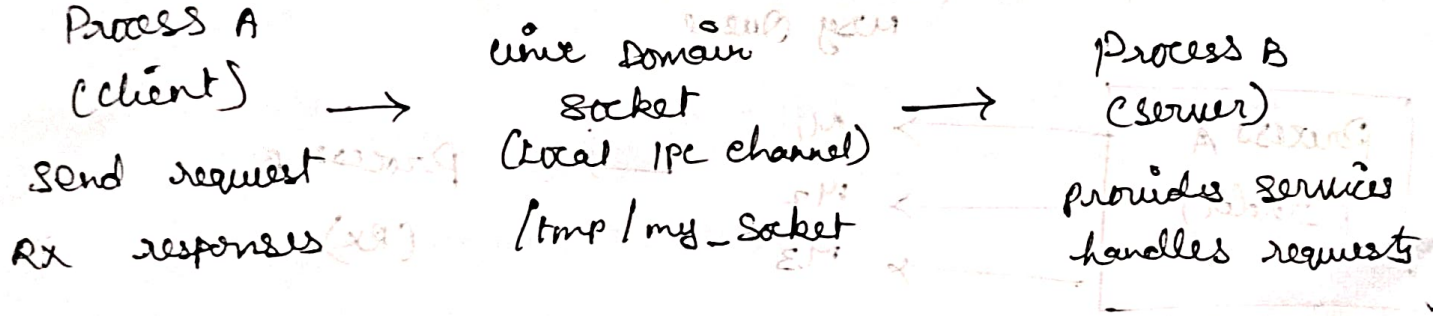
Function:

mq_open() → open/create
mq_send() → send msg
mq_receive() → Rx msg
mq_close() → close queue
mq_unlink() → remove queue

Unix domain Socket

It is a Socket mechanism used for commn b/w processes on the same Linux S/m.

It provides a familiar Socket interface without requiring n/w commn.



N/w Sockets

used for commn across different machines (Eg. over TCP/IP)

Unix domain Sockets used for commn within the same machine (local IPC)

It follows the same Socket API as n/w Sockets. A typical commn follows the client-server model

Common Socket API used

1) socket() → Create a unix domain Socket
2) bind() → Bind the socket to a file path
3) listen() → Listen for incoming connections
4) accept() → Accept a connection from a client
5) connect() → connect to the server Socket
6) send()/recv() → send & Rx data
7) close() → close the Socket

IPC mechanisms - Quick comparison

Features	PIPE	FIFO	Shared memory	msg Queue	Unix Domain Socket
1) Comm model	Byte stream (unidirection)	Byte stream → "	shared address space (Read/write)	Discrete msg (unidirection)	Stream or datagram (client-server)
2) Connection type	B/w Related Process	B/w unrelated Process	B/w Processes (through shared region)	B/w unrelated Processes	Client-server or peer to peer
3) Data transfer	Continuous byte stream → "		Direct read/write in memory	complete msg	Byte stream or datagram
4) Reliability	Reliable → "		Depends on synchronization	Reliable (msg stored)	Reliable with SOCK_STREAM
5) Performance	moderate → "		Very high	moderate	High
6) Best for	Simple parent child comm	comm b/w unrelated Process	high speed data sharing	Events, cmds, notification	Client-server request-response application
7) Key APIs / Calls	pipe(), read(), write()	mkfifo(), open() read(), write()	shm_open(), mmap() shm_unlink(), ftruncate()	mq_open(), mq_send(), mq_receive() mq_close()	socket(), bind(), listen() accept(), connect(), send(), RECV()